

SESSION 8

EXPERIMENT 15 — PROCESS MEMORY LAYOUT & DYNAMIC MEMORY ALLOCATION

Aim

To study the **process memory layout** and demonstrate dynamic memory allocation using `malloc()`, `calloc()`, `realloc()`, and `free()` in C.

Theory

A process is divided into different memory segments. The major parts of a process memory layout are:

1. Text Segment

Contains the executable program instructions/code.

2. Data Segment

Contains initialized global and static variables.

3. BSS Segment

Contains uninitialized global and static variables.

4. Heap

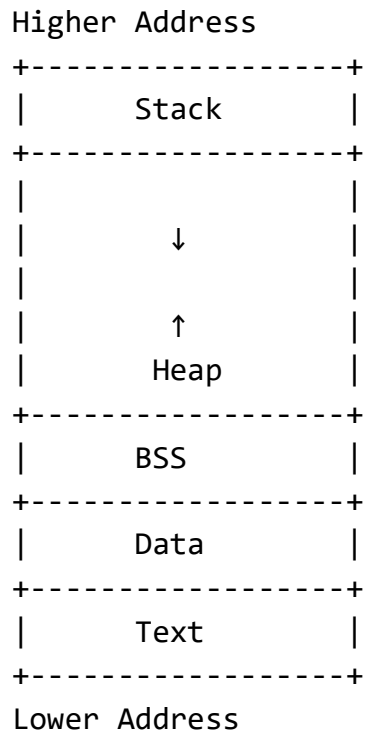
Used for **dynamic memory allocation** during program execution.

Functions such as `malloc()`, `calloc()`, and `realloc()` allocate memory from the heap.

5. Stack

Contains local variables, function parameters and function call information.

A simplified memory layout is:



Dynamic Memory Allocation

`malloc()`

Allocates a block of memory without initializing it.

```
ptr = malloc(size);
```

`calloc()`

Allocates memory and initializes it to zero.

```
ptr = calloc(n, size);
```

`realloc()`

Changes the size of an already allocated memory block.

```
ptr = realloc(ptr, new_size);
```

free()

Releases dynamically allocated memory.

```
free(ptr);
```

Algorithm

1. Declare global and local variables.
2. Display their memory addresses.
3. Allocate memory using `malloc()`.
4. Allocate memory using `calloc()`.
5. Resize allocated memory using `realloc()`.
6. Display the allocated memory addresses.
7. Release memory using `free()`.
8. Terminate the program.

C Program

```
C memory_layout.c > main()
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  int global_data = 100;
5  int global_bss;
6
7  void sample_function() {
8      printf("Inside sample function.\n");
9  }
10
11 int main() {
12
13     int local_variable = 10;
14
15     int *malloc_ptr;
16     int *calloc_ptr;
17
18     printf("Process Memory Layout\n");
19     printf("-----\n");
20
21     // Text segment
22     printf("Text segment (function): %p\n",
23           (void *)sample_function);
24
25     // Data segment
26     printf("Data segment (global initialized): %p\n",
27           (void *)&global_data);
28
29     // BSS segment
30     printf("BSS segment (global uninitialized): %p\n",
31           (void *)&global_bss);
32
33     // Stack
34     printf("Stack (local variable): %p\n",
35           (void *)&local_variable);
36
37     // malloc
38     malloc_ptr = (int *)malloc(5 * sizeof(int));
39
40     if (malloc_ptr == NULL) {
```

```
40     if (malloc_ptr == NULL) {
41         printf("malloc failed.\n");
42         return 1;
43     }
44
45     printf("Heap (malloc): %p\n",
46         (void *)malloc_ptr);
47
48     // calloc
49     calloc_ptr = (int *)calloc(5, sizeof(int));
50
51     if (calloc_ptr == NULL) {
52         printf("calloc failed.\n");
53         free(malloc_ptr);
54         return 1;
55     }
56
57     printf("Heap (calloc): %p\n",
58         (void *)calloc_ptr);
59
60     // realloc
61     malloc_ptr = (int *)realloc(malloc_ptr,
62         10 * sizeof(int));
63
64     if (malloc_ptr == NULL) {
65         printf("realloc failed.\n");
66         free(calloc_ptr);
67         return 1;
68     }
69
70     printf("Heap after realloc: %p\n",
71         (void *)malloc_ptr);
72
73     // Free memory
74     free(malloc_ptr);
75     free(calloc_ptr);
76
77     printf("\nDynamic memory released successfully.\n");
78
```

```

66     free(calloc_ptr);
67     return 1;
68 }
69
70 printf("Heap after realloc: %p\n",
71        (void *)malloc_ptr);
72
73 // Free memory
74 free(malloc_ptr);
75 free(calloc_ptr);
76
77 printf("\nDynamic memory released successfully.\n");
78
79 return 0;
80 }

```

OUTPUT:

```

chakrika@Chakrika:/mnt/c/Users/panak/Downloads/OSSP PRACTICALS$ gcc memory_layout.c -o memory_
layout
chakrika@Chakrika:/mnt/c/Users/panak/Downloads/OSSP PRACTICALS$ ./memory_layout
Process Memory Layout
-----
Text segment (function): 0x5d97c9d09209
Data segment (global initialized): 0x5d97c9d0c010
BSS segment (global uninitialized): 0x5d97c9d0c018
Stack (local variable): 0x7ffdeaa4b7d4
Heap (malloc): 0x5d97d63e06b0
Heap (calloc): 0x5d97d63e06d0
Heap after realloc: 0x5d97d63e06f0

Dynamic memory released successfully.
chakrika@Chakrika:/mnt/c/Users/panak/Downloads/OSSP PRACTICALS$

```

Result

Thus, the process memory layout was successfully studied and dynamic memory allocation was demonstrated using `malloc()`, `calloc()`, `realloc()`, and `free()`. The addresses of different memory segments were successfully observed.

EXPERIMENT 16 — COPY-ON-WRITE AND MEMORY ERRORS

This one has **two important parts** from your syllabus:

1. Copy-on-Write using `fork()`
2. Common memory errors in Linux

PART A — COPY-ON-WRITE

Aim

To demonstrate the **Copy-on-Write (COW)** mechanism using the `fork()` system call.

Theory

When a process calls `fork()`, the child process initially gets a copy of the parent's address space.

However, Linux does **not immediately copy all the memory**.

Instead, the parent and child initially share the same physical memory pages.

These pages are marked so that if either process tries to modify a page, the operating system creates a separate copy of that page.

This mechanism is called **Copy-on-Write (COW)**.

Why COW is useful

Without COW:

`fork()`
↓

Copy entire memory immediately

↓

High memory usage

With COW:

fork()

↓

Share memory pages

↓

Copy only when modified

↓

Lower memory usage

Algorithm

1. Create a variable in the parent process.
2. Allocate a memory block.
3. Store some data in it.
4. Call `fork()`.
5. Both parent and child initially see the same data.
6. Modify the memory in the child.
7. Display the child's modified value.
8. Display the parent's original value.
9. Observe that modifications are independent due to COW.

COW Program

```
C cow.c > main()
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <unistd.h>
4  #include <sys/wait.h>
5
6  int main() {
7
8      int *data;
9
10     data = (int *)malloc(sizeof(int));
11
12     if (data == NULL) {
13         perror("malloc");
14         return 1;
15     }
16
17     *data = 100;
18
19     printf("Before fork():\n");
20     printf("Parent value = %d\n", *data);
21
22     pid_t pid = fork();
23
24     if (pid < 0) {
25         perror("fork");
26         free(data);
27         return 1;
28     }
29
30     if (pid == 0) {
31
32         printf("\nChild process:\n");
33         printf("Value before modification = %d\n", *data);
34
35         *data = 200;
36
37         printf("Value after modification = %d\n", *data);
38
39         free(data);
40         exit(0);
    }
```

```

29
30     if (pid == 0) {
31
32         printf("\nChild process:\n");
33         printf("Value before modification = %d\n", *data);
34
35         *data = 200;
36
37         printf("Value after modification = %d\n", *data);
38
39         free(data);
40         exit(0);
41     }
42
43     else {
44
45         wait(NULL);
46
47         printf("\nParent process:\n");
48         printf("Value after child modification = %d\n",
49             *data);
50
51         free(data);
52     }
53
54     return 0;
55 }

```

OUTPUT:

```

chakrika@Chakrika:/mnt/c/Users/panak/Downloads/OSSP PRACTICALS$ gcc cow.c -o cow
chakrika@Chakrika:/mnt/c/Users/panak/Downloads/OSSP PRACTICALS$ ./cow
Before fork():
Parent value = 100

Child process:
Value before modification = 100
Value after modification = 200

Parent process:
Value after child modification = 100
chakrika@Chakrika:/mnt/c/Users/panak/Downloads/OSSP PRACTICALS$

```

PART B — MEMORY ERRORS IN C

Your syllabus mentions:

- Buffer Overflow
- Dangling Pointer
- Memory Leak
- Use-after-Free
- Double Free

These are important **viva/theory concepts**.

1. Buffer Overflow

Occurs when a program writes beyond the allocated size of a buffer.

Example:

```
char name[5];
```

Trying to store more than the available space can cause a buffer overflow.

2. Dangling Pointer

A pointer becomes dangling when the memory it points to has already been released.

```
int *ptr = malloc(sizeof(int));
```

```
free(ptr);
```

```
// ptr is now a dangling pointer
```

3. Memory Leak

A memory leak occurs when dynamically allocated memory is not released.

```
int *ptr = malloc(100 * sizeof(int));
```

```
/* memory is never freed */
```

The memory remains allocated until the process terminates.

4. Use-after-Free

Occurs when a program accesses memory after it has already been released.

```
int *ptr = malloc(sizeof(int));
```

```
free(ptr);
```

```
/* ptr should not be used here */
```

5. Double Free

Occurs when the same memory is released more than once.

```
free(ptr);
```

```
free(ptr);
```

This can cause undefined behavior.

Safe Memory-Error Demonstration

For your practical, I recommend **not intentionally crashing the program** just to demonstrate these errors. Instead, you can show how they are prevented.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main() {

    char *buffer;
    int *ptr;

    printf("Memory Error Prevention Demonstration\n");
    printf("-----\n");

    // Prevent buffer overflow
    buffer = (char *)malloc(20 * sizeof(char));

    if (buffer == NULL) {
        printf("Memory allocation failed.\n");
        return 1;
    }

    strcpy(buffer, "Hello");

    printf("Buffer: %s\n", buffer);

    // Use allocated memory
    ptr = (int *)malloc(sizeof(int));

    if (ptr == NULL) {
        free(buffer);
        return 1;
    }

    *ptr = 100;

    printf("Allocated value: %d\n", *ptr);
```

```

    *ptr = 100;

    printf("Allocated value: %d\n", *ptr);

    // Release memory exactly once
    free(ptr);
    ptr = NULL;

    free(buffer);
    buffer = NULL;

    printf("Memory released safely.\n");
    printf("Pointers set to NULL to prevent dangling-pointer use.\n");

    return 0;
}

```

OUTPUT:

```

chakrika@Chakrika:/mnt/c/Users/panak/Downloads/OSSP PRACTICALS$ gcc memory_errors.c -o memory_errors
chakrika@Chakrika:/mnt/c/Users/panak/Downloads/OSSP PRACTICALS$ ./memory_errors
Memory Error Prevention Demonstration
-----
Buffer: Hello
Allocated value: 100
Memory released safely.
Pointers set to NULL to prevent dangling-pointer use.
chakrika@Chakrika:/mnt/c/Users/panak/Downloads/OSSP PRACTICALS$

```

Result

Thus, the Copy-on-Write mechanism was successfully demonstrated using `fork()`. It was observed that the parent and child initially shared the same memory contents, and a separate copy was created when the child modified the data. Common Linux memory errors such as buffer overflow, dangling pointers, memory leaks, use-after-free, and double free were also studied along with their prevention.